

ITL351 – Computer Networks Lab

Network Protocol Stack Simulator

Specification & Submission Report

Name	Enroll No.
Priyanshu Dayal	2023BITE056
Gulaba Ram	2023BITE084
Pranav Abani	2023BITE078

GitHub Repository: github.com/Gulabkt98/CN_simulator

Language & Libraries Used

Language: C++ (C++17) — Object-oriented and procedural paradigms.

Standard Library: `<string>`, `<vector>`, `<map>`, `<queue>`, `<sstream>`, `<cstdint>` — for routing tables, ARP cache, MAC tables, port handlers, and packet queues.

Data Structures: `std::map` and `std::unordered_map` for MAC/ARP/routing tables; `std::vector` for connection lists and sliding windows; `std::queue` for ACK buffers.

I/O Representation: Console-based text output with structured tagged print statements for clear protocol-level tracing.

Project Overview

This project implements a fully object-oriented Network Protocol Stack Simulator in C++ (C++17). The simulator models networking concepts layer by layer across all three submissions — Physical and Data Link layers in Submission 1, the Network layer in Submission 2, and Transport and Application layers in Submission 3. All devices, connections, and protocol behaviors are represented as C++ classes, enabling realistic multi-hop simulation of routing, address resolution, port-based process communication, and application services without any real network hardware.

Submission 2 – Network Layer

Objective

Extend the simulator with Layer 3 functionality: IPv4 CIDR addressing, static routing with longest-prefix matching, ARP-based MAC address resolution, and multi-hop packet forwarding through routers.

Implemented Components

Component	Description
IPv4CIDR	Parses IPv4 addresses in dotted-decimal format. Supports CIDR notation (/8 to /32), computes subnet masks, and performs network membership testing via bitwise AND.
RoutingTable	Maintains CIDR routes with next-hop information. Implements longest-prefix matching so more specific routes always take priority. Supports add, remove, lookup, and display.
NetworkPacket	IP-layer packet carrying source/destination IP, TTL (starts at 30, decrements per hop), hop count, and payload. Serialised to a string for encapsulation inside Data Link frames.
ARP	ARP cache mapping IP addresses to MAC addresses. Supports cache lookup and entry addition. ARP messages carry sender/target IP–MAC pairs with REQUEST and REPLY types.

Component	Description
Router	Multi-hop forwarding device. Deserialises incoming packets, checks TTL (drops at 0), performs longest-prefix route lookup, decrements TTL, creates a new Frame with updated MACs, and learns reverse paths for ACK routing.

Test Cases

- Test 11 – CIDR IPv4 Parsing: Verifies correct subnet membership for addresses against multiple CIDR prefixes (/24, /25, /16). Membership tests pass for in-range IPs and correctly fail for out-of-range ones.
- Test 12 – Longest-Prefix Matching: A routing table is configured with overlapping /0, /24, and /25 routes. Packets are routed to the most specific matching prefix (e.g. 10.0.0.50 matches /25 over /24).
- Test 13 – ARP Protocol: Exercises ARP cache add/lookup operations. Cached IPs return correct MACs; uncached IPs correctly report NOT FOUND.
- Test 8–9 – Multi-Hop End-to-End Routing: End device A (10.0.0.1) sends a message to end device B (10.0.0.2) across two hub-based subnets connected by Router R1. TTL is decremented at the router. The reply is routed back via the reverse path learned by R1.

Assumptions

- All IP addresses are valid IPv4 in dotted-decimal format.
- Routing tables are configured statically at simulation setup.
- ARP cache is pre-populated; no broadcast ARP request simulation.
- TTL starts at 30 and is decremented at each router hop; packets with TTL = 0 are dropped.
- Longest-prefix matching selects the route with the highest prefix length among all matching entries.

Submission 3 – Transport & Application Layers

Objective

Extend the simulator with Layer 4 (Transport) and Layer 7 (Application) functionality including port-based process-to-process communication, sliding-window flow control at the transport layer, and two application-layer services demonstrating full-stack encapsulation.

Implemented Components

Component	Description
TransportSegment	Transport-layer PDU with 16-bit source and destination ports, sequence number, acknowledgement number, isACK flag, and string payload. Serialised into NetworkPacket payloads for IP-level delivery.
TransportLayer	Port-based demultiplexing on each EndDevice. Service handlers are registered per destination port. On segment arrival the correct handler is invoked by port number.
Echo Service (Port 7)	Implements the Echo protocol. Receives a TransportSegment on port 7 and replies with the identical payload back to the originating IP and port, verifying end-to-end connectivity.
File Transfer (Port 21)	Simplified FTP-style service on port 21. Supports PUT filename content to store a file and GET filename to retrieve it. Maintains a per-session in-memory file store and replies with FILE_STORED or file contents.
Sliding Window (Transport)	Go-Back-N (N = 4) reused from the Data Link layer and applied at the transport layer, providing reliable ordered delivery of TransportSegments between processes.

Test Cases

- Test 14 – Echo Service: Host A sends a TransportSegment with destination port 7 to Host B. The segment traverses the full stack (Application → Transport → Network → Data Link → Physical → Router → Physical → Network → Transport → Application). The Echo Service on B demultiplexes by port 7 and echoes the payload back to A on port 5000.
- Test 15 – File Transfer Service: Host A sends PUT|document.txt|<content> to port 21 on Host B. The FTP service stores the file and replies FILE_STORED. A subsequent GET retrieves the content. Multi-layer encapsulation is verified at each layer.

Assumptions

- Well-known ports: 7 (Echo), 21 (FTP). Ephemeral client ports: 5000 and above.
- Sliding window at transport layer reuses Go-Back-N from Data Link layer; window size N = 4 (configurable at runtime).
- Service handlers are registered before simulation begins; unregistered ports log a port-unreachable message.
- FTP service maintains an in-memory file store; no actual disk I/O is performed.

How to Run the Code

Requirements: C++17 compatible compiler (g++ 9 or higher). No third-party packages required.

- Clone the repository: `git clone https://github.com/Gulabkt98/CN_simulator`
- Navigate to the directory: `cd CN_simulator`
- Compile: `g++ main.cpp devices/*.cpp application/*.cpp network/*.cpp -std=c++17 -o simulator.exe`
- Run: `./simulator.exe` (Windows) or `./simulator.exe` (Linux/macOS)

Each test prints a detailed trace of packet flow and protocol events. Tests 1–7 cover CSMA/CD and Go-Back-N. Tests 8–9 cover multi-hop routing. Tests 10–13 cover Bridge topology, CIDR, longest-prefix routing, and ARP. Tests 14–15 cover Transport layer echo and file transfer services.

Output Format

All output is console-based. Each event is tagged with a bracketed label:

- [ARP] – ARP cache lookup or entry addition.
- [ROUTER] – Routing decision, TTL decrement, and next-hop selection.
- [CIDR] / [Routing] – Subnet membership test and route lookup result.
- [Transport] – Port demultiplexing and segment delivery.
- [Echo Service] / [File Transfer] – Application service events and replies.
- [Application Layer] – Message composed or delivered at the application layer.
- [RESULT] – Test pass/fail summary.

Add-ons Implemented

- CIDR classless addressing with address aggregation support.
- IPv4 datagram serialisation — network packets encoded as structured strings and encapsulated inside Data Link frames.
- Reverse path learning: the Router records incoming port/device per source IP, enabling correct ACK routing without static reverse routes.
- Selective Repeat protocol implemented as an alternative to Go-Back-N at the transport layer.

- Topology analysis: collision domain and broadcast domain reporting after every topology change.

References

- Guidance and mentorship by Dr Iqra Altaf Gilani, Department of IT, NIT Srinagar.
- Class notes: Network Layer, IP Addressing & CIDR, ARP, Transport Layer, and Application Layer Services.
- OpenAI. (2024). ChatGPT [Large language model]. Used for code assistance and conceptual clarification. <https://chat.openai.com>